GOOGLE | 1POINT3ACRES, PRACHUB | TOPIC 16 OF 42 | CORE 90 CHAPTER TOPIC

Core 90 - Recursion and backtracking

2 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Recursion and backtracking | 2 questions | tap a title to jump to it

M 1. Mahjong Winning Hand Detection (14 Tiles) **1Point3Acres 2026-10-09**

M 2. Count Prefix-Matching Simple Paths in a Labeled Graph **PracHub 2026-08-09**

STUDY PATH: question 79 of 167 (Core 90)

Priority score 6.0 | Recursion and backtracking | asked 11 days ago | frequency Low

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 1 of 2

Mahjong Winning Hand Detection (14 Tiles)

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 210/228**

ABOUT THIS QUESTION

Topic: Core 90 - Recursion and backtracking

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

APPROACH

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

HOW TO THINK ABOUT IT

A winning hand is one pair plus four melds (a triplet of identical tiles or a run of three consecutive ranks in one suit). Try every tile as the pair, then repeatedly take the smallest remaining tile: it can only be covered by a triplet of itself or by a run that starts at it, so there are at most two choices at each step and the search is tiny for 14 tiles.

HOW TO THINK ABOUT IT

Basis: Written for this question; the pair-plus-melds check is the same backtracking as LeetCode 846, Hand of Straights, plus a triplet option.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(tiles):
    for t in set(tiles):
        if tiles.count(t) >= 2:
            rest = remove(tiles, [t, t])
            if all_melds(rest):
                return True
    return False
def all_melds(tiles):
    if not tiles:
        return True
    first = min(tiles)
    for meld in (triple(first), run(first)):
        if contains(tiles, meld):
            if all_melds(remove(tiles, meld)):
                return True
    return False
```

TIME	SPACE
roughly C(12,3) * C(9,3) ... roughly C(12,3) * C(9,3) * C(6,3) per pair	**O(1)**

WHY IT IS SLOW

Tries every set of three tiles at every step. The smallest remaining tile can only be in a triplet of itself or a run starting at it, so at most two choices remain.

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

TIME	SPACE
O(1) for 14 tiles (about 2^4 branches per pair)	**O(distinct tiles)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A variant needs only two melds plus a pair (8 tiles of 14).

Ask what happens to the other six tiles before coding: if they are free, check whether any 8 tiles form a pair and two melds with the same search over subsets of the 14. The rule set is intentionally vague, so confirm melds, honours and flowers first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Mahjong Winning Hand Detection (14 Tiles)
#
# Mountain View onsite coding round 1: given a hand of 14
# tiles, decide whether it is a winning mahjong hand. The
# candidate must clarify the rule set (typically: 4 melds
# of three identical tiles or three consecutive ranks + 1
# pair). Input: array of 14 integers representing tile
# values (clarify suit / range during the interview). A
# winning hand consists of: One pair (two identical
# tiles), and Four melds, each being either: A triplet (3
# identical tiles), or A run (3 consecutive ranks in the
# same suit). Minority variant: A newer version instead
# specifies one pair plus two melds while retaining a
# 14-tile input. Because that accounts for only eight
# tiles, clarify the intended meld count and how the
# remaining six tiles should be handled before coding.
# Output: boolean.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A variant needs only two melds plus a pair (8 tiles
# of 14).
#
# INTERVIEW NOTES
# A winning hand is one pair plus four melds (a triplet of
# identical tiles or a run of three consecutive ranks in
# one suit). Try every tile as the pair, then repeatedly
# take the smallest remaining tile: it can only be covered
# by a triplet of itself or by a run that starts at it, so
# there are at most two choices at each step and the
# search is tiny for 14 tiles.
#
# BRUTE FORCE (say it first): Choose any three tiles for
# every meld
# For each pair, try every set of three remaining tiles
# that forms a meld and recurse. Correct and the oracle;
# it ignores that the smallest tile must be used first.
# Time roughly C(12,3) * C(9,3) * C(6,3) per pair. Space
# O(1).


import itertools
from collections import Counter


# BRUTE FORCE (say it first): Choose any three tiles for
```

```
# every meld | time roughly C(12,3) * C(9,3) * C(6,3) per
# pair
def mahjong_brute(hand):
    def is_meld(t):
        t = sorted(t)
        same = t[0] == t[1] == t[2]
        run = (
            t[0] // 10 == t[1] // 10 == t[2] // 10 < 3
            and t[1] == t[0] + 1
            and t[2] == t[0] + 2
        )
        return same or run

    def melds(tiles):
        if not tiles:
            return True
        for idx in itertools.combinations(
            range(len(tiles)), 3
        ):
            if is_meld(
                [tiles[i] for i in idx]
            ) and melds(
                [
                    t
                    for i, t in enumerate(tiles)
                    if i not in idx
                ]
            ):
                return True
        return False

    for i, j in itertools.combinations(
        range(len(hand)), 2
    ):
        if hand[i] == hand[j] and melds(
            [
                t
                for k, t in enumerate(hand)
                if k not in (i, j)
            ]
        ):
            return True
    return False
```

```
# HELPERS for the base question
def is_run_start(tile, count):
    # numbered suits are 0..2 with ranks 1..9; the tile
    # id is suit * 10 + rank. Honours (30+) never run.
    ok = tile < 30 and tile % 10 <= 7
    nxt, nxt2 = count.get(tile + 1), count.get(tile + 2)
    return ok and nxt and nxt2


def melds_left(count, need):
    # can the tiles in `count` form exactly `need` melds?
    live = [t for t in sorted(count) if count[t]]
    tile = live[0] if live else None
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3
        if ok:
            return True
    # ... or starts a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1
        return ok
    return False


# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# Idea: Tiles are encoded as suit * 10 + rank for the
# numbered suits (ranks 1 to 9, suits 0 to 2) and 30 and
# above for honours, which never form runs.
# FOLLOW-UP 1: A variant needs only two melds plus a pair
# (8 tiles of 14).
# Answer: Ask what happens to the other six tiles before
# coding: if they are free, check whether any 8 tiles form
# a pair and two melds with the same search over subsets
```

```
# of the 14.
def is_winning_hand(hand, melds=4):
```

```
    # FOLLOW-UP 1: melds=2 handles the 8-tile variant
    # one pair plus `melds` melds uses every tile
    if len(hand) != 2 + 3 * melds:
        return False
    count = Counter(hand)
    # try every tile value as the pair
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False


if __name__ == "__main__":
    win = [1, 1, 1, 2, 3, 4, 11, 12, 13,
           21, 22, 23, 31, 31]
    assert is_winning_hand(win)
    # 2, 3, 5 is not a run
    lose = [1, 1, 1, 2, 3, 5, 11, 12, 13,
            21, 22, 23, 31, 31]
    assert not is_winning_hand(lose)
    # honours only make triplets or the pair
    honours = [30, 30, 31, 31, 31, 32, 32, 32, 33, 33, 33,
               34, 34, 34]
    assert is_winning_hand(honours)
    # the smaller variant: a pair and two melds (8 tiles)
    assert is_winning_hand([5, 5, 1, 2, 3, 7, 7, 7], 2)
    # wrong tile count is never a win
    assert not is_winning_hand(win[:13])
```

NOTES FROM CANDIDATES

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:

Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.

For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

PREPARATION

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY PATH: question 80 of 167 (Core 90)

Priority score 4.7 | Recursion and backtracking | asked 72 days ago

MEDIUM | PracHub | Technical Screen | 2 of 2

Count Prefix-Matching Simple Paths in a Labeled Graph

POSTED 2026-08-09 | 72 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 42.5** | **30 DAYS 0** | **PRIORITY 83/228**

ABOUT THIS QUESTION

Topic: Core 90 - Recursion and backtracking

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #79 Word Search

LeetCode match: No direct match; related LC 79 Word Search (inferred from overview)

Posted on PracHub: 2026-08-09 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/count-prefix-matching-simple-paths-in-a-labeled-graph

QUESTION

Overview: Count distinct simple graph paths whose labels match every nonempty prefix of a query word. Paths may start anywhere but cannot revisit a vertex, so the task rewards subset-state search and careful counting over a small labeled graph.

APPROACH

DFS/backtracking from every start vertex, extending a path only while the labels match the next query character and the vertex is unvisited (visited bitmask); count paths for every prefix length.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the graph is directed, whether a path is counted as a vertex sequence, whether paths of every prefix length are counted (the statement says every nonempty prefix), and the size (small, since the state is a visited subset).

BRUTE FORCE TO SAY OUT LOUD

Enumerate all simple paths and then test labels. Exponential and wasteful.

OPTIMAL IDEA

DFS with backtracking from every start vertex. Extend a path only if the next vertex is unvisited (a bitmask of visited vertices) and its label equals the next character of the query word. Every time the path length is L, count one for prefix length L. The pruning on labels keeps the search small. Time depends on the number of matching paths, bounded by n * d^(L-1).

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

A query of length 1 (count of vertices with that label), a graph with a cycle (simple paths may not revisit), a label that appears nowhere, repeated letters in the query.

LIKELY FOLLOW-UPS

Longer queries (memoize on (vertex, mask, position) only helps if counting, not listing), or undirected edges.

PITFALLS

Counting a path that revisits a vertex. Counting an undirected path twice (once in each direction) without agreeing on it.

HOW TO THINK ABOUT IT

A path is only worth extending while its labels still match the query, so prune on the label before going deeper. Distinctness is tracked with a visited set (a bitmask for small graphs). Count at every depth, since each prefix length has its own answer.

Basis: Written; backtracking on a small labelled graph. Assumed rules (confirm them): directed edges; a path is a sequence of distinct vertices; for each L from 1 to the length of the query, count the paths of L vertices whose labels spell the first L letters of the query.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
def dfs(path):
    global count
    if len(path) <= len(query):
        if matches_prefix(path):
            count += 1
    for nxt in graph[path[-1]]:
        if nxt not in path:
            dfs(path + [nxt])
for v in vertices:
    dfs([v])
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

It explores paths whose first label already disagrees with the query. Extending a path only when the next label matches the next query letter prunes those branches.

OPTIMAL SOLUTION · DFS with label pruning and a visited bitmask

TIME	SPACE
O(number of matching paths * degree)	O(path)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges are undirected.

Add every edge in both directions, as above. A path is then counted once per direction unless you divide the counts for length >= 2 by 2 (a path and its reverse are different sequences only if the query is not a palindrome, so say how you count).

FOLLOW-UP 2 Count only the paths that spell the whole query.

Return counts[-1] instead of the list. For long queries on bigger graphs, memoise on (vertex, depth) if repeated vertices are allowed (walks, not simple paths); for simple paths the visited set prevents memoisation.

FOLLOW-UP 3 The graph has up to 100,000 vertices.

Simple paths are exponential, so ask whether walks are allowed. For walks a DP over (depth, vertex) counts them in O(L * E), the standard way to handle large graphs.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Prefix-Matching Simple Paths in a
# Labeled Graph
#
# Count distinct simple graph paths whose labels match
# every nonempty prefix of a query word. Paths may start
# anywhere but cannot revisit a vertex, so the task
# rewards subset-state search and careful counting over a
# small labeled graph.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges are undirected.
# 2. Count only the paths that spell the whole query.
# 3. The graph has up to 100,000 vertices.
#
# INTERVIEW NOTES
# A path is only worth extending while its labels still
# match the query, so prune on the label before going
# deeper. Distinctness is tracked with a visited set (a
# bitmask for small graphs). Count at every depth, since
# each prefix length has its own answer.
#
# BRUTE FORCE (say it first): Enumerate all simple paths,
# then test the labels
# List every simple path of every length and compare its
# labels with each prefix of the query. Correct and the
# oracle; it explores paths whose first label already
# disagrees. Time exponential. Space O(path).


from collections import defaultdict


# BRUTE FORCE (say it first): Enumerate all simple paths,
# then test the labels | time exponential
def prefix_paths_brute(labels, edges, query):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    counts = [0] * len(query)

    def go(path):
        word = "".join(labels[v] for v in path)
        if (
            len(path) <= len(query)
            and word == query[: len(path)]
        ):
            counts[len(path) - 1] += 1
        if len(path) < len(query):
            for w in adj[path[-1]]:
                if w not in path:
                    go(path + [w])

    for s in range(len(labels)):
        go([s])
    return counts


# BASE QUESTION (optimal): DFS with label pruning and a
# visited bitmask
# Idea: Start from every vertex whose label equals the
# first query letter. Extend to an unvisited neighbour
# only if its label equals the next query letter, adding
# one to the count for the current length at each step.
def prefix_paths(labels, edges, query, undirected=False):
    # counts[d]: simple paths whose first d + 1 labels
    # spell
    # the first d + 1 letters of the query
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        # FOLLOW-UP 1: undirected=True adds both
        # directions
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)

    def go(u, depth, mask):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            # a simple path never revisits a vertex
            if mask >> w & 1:
                continue
            if labels[w] == query[depth + 1]:
                go(w, depth + 1, mask | 1 << w)

    # a path may start at any vertex with the first letter
    for s in range(len(labels)):
        if query and labels[s] == query[0]:
            go(s, 0, 1 << s)
    return counts


# VARIANT: walks may revisit vertices
def prefix_walks(labels, edges, query):
    # walks may revisit vertices: a DP over (depth,
    # vertex)
    # costs O(L * E) and works for very large graphs
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    ways = {v: 1 for v in range(len(labels))
            if query and labels[v] == query[0]}
    counts = [sum(ways.values())] if query else []
    for ch in query[1:]:
        nxt = defaultdict(int)
        for u, n in ways.items():
            for w in adj[u]:
```

```
                if labels[w] == ch:
                    nxt[w] += n
        ways = nxt
        counts.append(sum(ways.values()))
    return counts


# FOLLOW-UP 2: Count only the paths that spell the whole
# query.
# Answer: Return counts[-1] instead of the list. For long
# queries on bigger graphs, memoise on (vertex, depth) if
# repeated vertices are allowed (walks, not simple paths);
# for simple paths the visited set prevents memoisation.
def whole_query_paths(labels, edges, query,
                      undirected=False):
    # only the paths that spell the entire query
    both = undirected
    counts = prefix_paths(labels, edges, query, both)
    return counts[-1] if counts else 0


if __name__ == "__main__":
    labels = ["a", "b", "a", "b"]
    edges = [(0, 1), (1, 2), (2, 3), (3, 0)]
    # "aba": directed square a>b>a>b>...
    assert prefix_paths(labels, edges, "aba") == [2, 2, 2]
```

```
    assert whole_query_paths(labels, edges, "aba") == 2
    # undirected: every edge works both ways
    und = prefix_paths(labels, edges, "ab", True)
    assert und == [2, 4]
    # a walk may reuse vertices, a simple path may not
    walks = prefix_walks(labels, edges, "abab")
    assert walks == [2, 2, 2, 2]
    five = prefix_paths(labels, edges, "ababa")
    assert five == [2, 2, 2, 2, 0]
    assert prefix_paths(labels, edges, "") == []
```

FUNCTION AND TEST CASES

Signature

```
def count_prefix_paths(labels: str, edges: list[list[int]]
, word: str) -> list[int]:
```

Sample call

```
count_prefix_paths('roteoreote', [[0, 1], [0, 2], [0, 3],
[1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')
```

Test cases

Input	Expected
`('roteoreote', [[0, 1], [0, 2], [0, 3], [1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')`	`[2, 2, 1, 1, 0]`
`('a', [], 'aa')`	`[1, 0]`